

Se tratando de um relacionamento entre Pessoa e Entidade, este relacionamento é de N:1:



Tomando como base a biblioteca Spring Data JDBC que possui uma estrutura lógica de agregação entre entidades, alguns cenários de mapeando entre classes seriam possíveis:

- Declarar na classe Pessoa um atributo de instância do tipo Nacionalidade. Isso seria representado como um relacionamento 1:1 no banco de dados, com a tabela Nacionalidade possuindo uma coluna como chave primária e estrangeira apontando para a tabela Pessoa. Isso tornaria a Nacionalidade agregada à Pessoa e estaria sujeita a inserção e deleção em conjunto com Pessoa. **Não é isso que desejamos.**
- Declarar na classe Nacionalidade um atributo de instância do tipo lista de Pessoas com uma anotação `@MappedCollection`. Isso seria representado como um relacionamento 1:N (de Nacionalidade para Pessoa) no banco de dados, com a tabela Pessoa possuindo uma chave estrangeira apontando para Nacionalidade (e também uma coluna para ordenação). Isso tornaria a classe Pessoa agregada à Nacionalidade e estaria sujeita a inserção e deleção em conjunto com Pessoa. **Não é isso que desejamos.**
- Declarar na classe Pessoa um atributo do tipo inteiro que representasse o identificador da Nacionalidade. Isso seria representado como um relacionamento N:1 (de Pessoa para Nacionalidade) no banco de dados, com a tabela Pessoa possuindo uma chave estrangeira apontando para Nacionalidade. Desta forma a Nacionalidade **não** ficaria agregada à Pessoa, não estando sujeita a inserção e deleção em conjunto com Pessoa. **Isso é o que desejamos.**
 - Alternativamente pode-se declarar na classe Pessoa um atributo de instância do tipo Genérico `AggregateReference` com os tipos Nacionalidade e inteiro como argumentos do Genérico, informando opcionalmente a coluna do banco para armazenar identificador da Nacionalidade. Na prática a tabela Pessoa armazenará a chave estrangeira para Nacionalidade formando um relacionamento N:1 no banco de dados. **Porém as duas classes também NÃO ficarão agregadas.**

○

Em relação à forma de exibir a nacionalidade de uma lista de pessoas, têm-se as seguintes estratégias:

- Considerando o uso de repositórios das classes Pessoa e Nacionalidade (que estendem de *CrudRepository*): buscar a lista de pessoas do banco de dados, depois percorrer essa lista e buscar no banco de dados cada nacionalidade. **Isso NÃO é eficiente.**
 - Obs: ao utilizar a estratégia 1:N com anotação `@MappedCollection`, por padrão o spring data jdbc realiza uma query para a entidade principal e outra para retornar as instâncias da lista agregada.
- Uma vez que a tabela de Nacionalidade quase nunca muda, poderíamos utilizar Cache:
 - Nativo do Spring Boot: utilizar cache na consulta de Nacionalidade por Id no *NacionalidadeRepository* (que estende de *CrudRepository*). Ao percorrer a lista de pessoas e buscar a Nacionalidade, se ela já estiver sido carregada não é feita a consulta no banco de dados. (<https://docs.spring.io/spring-boot/reference/io/caching.html>). **Eficiente, porém nem tanto.**
 - Implementado manualmente: poderia ser implementado com um Bean do Spring que contivesse um `Map<Long,Nacionalidade>` através do `findAll` no *NacionalidadeRepository* (que estende de *CrudRepository*) com todas as Nacionalidades carregadas. Ao percorrer a lista de pessoas, buscar a Nacionalidade no Map pelo Id da Nacionalidade. **Isso é eficiente.**
- DTO: criar um record *PessoaComNacionalidadeDTO* (*Data Transfer Object*) e implementar no *PessoaRepository* (que estende de *CrudRepository*) um método com uma `@Query` que faça um join da tabela Pessoa com a tabela Nacionalidade e retorne uma lista de objetos *PessoaComNacionalidadeDTO*. Os campos trazidos na cláusula select do `@Query` precisam ter a mesma ordem e os mesmos nomes (com alias) que os atributos do record. **Isso é eficiente.**